

FEUP-L.EIC024-2024/2025-1S

Início terça-feira, 9 de setembro de 2025 às 11:33**Estado** Prova submetida**Data de
submissão:** terça-feira, 9 de setembro de 2025 às 13:33**Tempo gasto** 2 horas**Nota** **4,30** de um máximo de 20,00 (**21,5%**)

Informação

Este grupo de questões é relativo a um conjunto de filmes.

Cada filme tem um título e uma lista ordenada de atores.

Considere a seguinte estrutura de dados para representar um filme:

```
type Movie = (String, [String])
```

Exemplo de uma lista de filmes:

```
myMovies :: [Movie]
myMovies = [(("Titanic", ["Kate Winslet","Leonardo DiCaprio"]), ("Revolutionary Road",["Kate Winslet","Leonardo DiCaprio"]),
            ("Elektra ",["Jennifer Garner"]), ("The Great Gatsby",["Leonardo DiCaprio"])]
```

Assuma ainda a existência das seguintes funções:

- **delAll :: (Eq a) => a -> [a] -> [a]**, que recebe uma lista *l* e um elemento *x*, e devolve a sublista de *l* onde todos os elementos iguais a *x* foram removidos.
- **uniqueVals :: (Eq a) => [a] -> [a]**, que remove todos os elementos duplicados da lista de entrada, mantendo a ordem dos elementos.

Pergunta 1

Pontuou 1,000 de 1,000

Implemente a função **findActors :: [Movie] -> String -> Maybe [String]**, que recebe uma lista de filmes '*movies*' e o título de um filme '*title*', e devolve a lista dos atores correspondente ao filme com título '*title*' em '*movies*'.

Se o filme não existir, então deve retornar *Nothing*.

```
findActors :: [Movie] -> String ->
Maybe [String]
findActors [] _ = Nothing
findActors ((x, y):xs) movie
  | x == movie = Just y
```

Pergunta 2

Pontuou 1,000 de 1,000

Implemente a função **moviesWithActor :: [Movie] -> String -> [Movie]**, que recebe uma lista de filmes '*movies*' e o nome de um ator '*actor*', e devolve a sublista de '*movies*' dos filmes em que '*actor*' participou.

A sublista de saída deve preservar a ordem dos filmes em '*movies*'.

```
moviesWithActor :: [Movie] -> String
-> [Movie]
moviesWithActor [] _ = []
moviesWithActor ((x, y):xs) actor
  | actor `elem` y = (x, y) :
```

Pergunta 3

Pontuou 1,000 de 1,000

Implemente a função ***uniqueActors :: [Movie] → [String]***, que devolve uma lista com todos os atores da lista de filmes '***movies***', sem repetidos.

Os atores devem estar na lista de saída pela mesma ordem em que ocorrem pela primeira vez em '***movies***'.

```
uniqueActors :: [Movie] → [String]
uniqueActors [] = []
uniqueActors movie =
  uniqueVals(auxActors movie)
```

Pergunta 4

Pontuação 1,000

Implemente a função ***actorFrequencies :: [Movie] → [(String, Int)]***, que devolve uma lista de pares '*(nome de ator, numero de filmes em que participa)*' da lista de filmes '***movies***'.

Os atores devem estar na lista de saída pela mesma ordem em que ocorrem pela primeira vez em '***movies***' (ou seja, não precisa nem deve ordenar os atores pelo número de filmes).

Pergunta 5

Pontuação 0,500

Qual das seguintes afirmações sobre *typeclasses* está errada?

- ☐ a. *Ord* é uma *typeclass* para tipos que suportam o operador `<=`.
- ☐ b. *Eq* é uma subclasse de *Ord*.
- ☐ c. *Float* faz parte da *typeclass* *Fractional*.
- ☐ d. *Int* faz parte da *typeclass* *Integral*.
- ☐ e. Um tipo pode pertencer a três *typeclasses* diferentes.

Resposta correta: *Eq* é uma subclasse de *Ord*.

Pergunta 6

Pontuou 0,500 de 0,500

Indique um tipo possível para a seguinte expressão:

```
[ (n, x, x !! n) | n <- 1s]
```

- ☐ a. `[(Int, [Char], [Char])]`
- ☐ b. `[(Int], [String], String)`
- ☒ c. `[(Int, [Int], Int)]` ✓
- ☐ d. `[(Int, String, String)]`

Resposta correta:

```
[(Int, [Int], Int)]
```

Pergunta 7

Pontuação 1,000

Considere a seguinte função *funcX*:

```
funcX = foldl1 (\acc _ -> pred acc) 0
```

Reimplemente esta função usando a técnica que preferir e SEM usar funções da família dos *folds* (*foldr*, *foldl*, *foldr1*, *scanr*, ...).

Pergunta 8

Pontuação 1,000

Usando a função *zip* defina a função ***evenPos***, que dadas duas listas *xs* e *ys* (com tamanhos que podem ser diferentes) retorna a lista de pares de elementos que aparecem nas mesmas posições pares das listas, mas apenas se estes tiverem valores diferentes. Por exemplo:

```
evenPos [1,2,3,4,5] [5,4,3,2,1] = [(1,5),(5,1)]  
evenPos "banana" "batata" = [('n','t'),('n','t')]
```

Pergunta 9

Pontuação 1,000

Considere a seguinte série definida recursivamente da seguinte forma:

```
a_0 = 0  
a_1 = 1  
a_n = (3 * a_(n-1)) - 2 * a_(n-2) if n > 1
```

Defina a lista infinita *list* contendo todos os termos da série:

```
[0,1,-2,7,-20,61,-182,547,-1640,...]
```

Pergunta 10

Pontuação 2,000

Considere a seguinte definição de um tipo recursivo para representar expressões em lógica proposicional:

```
data Prop = Const Bool      -- constantes
          | Neg Prop         -- negação de uma sub-expressão
          | Conj Prop Prop   -- conjunção de duas sub-expressões
          | Disj Prop Prop   -- disjunção de duas sub-expressões
          | Impl Prop Prop   -- implicação de duas sub-expressões
```

Escreva uma definição recursiva da função **valor :: Prop → Bool** que calcula o valor lógico de uma expressão.

Informação

O IMDB disponibiliza informação sobre personagens, para além de atores e filmes/séries. Suponha que o seguinte conjunto de factos é ilustrativo de uma base de conhecimento, onde é indicado o nome da personagem, o nome do filme/série em que apareceu, e o nome do ator que a interpretou:

```
:-dynamic by/3.

% by(Character, Movie, Actor)
by(jackRyan, theSumOfAllFears, benAffleck).
by(cathyMuller, theSumOfAllFears, bridgetMoynahan).
by(jackRyan, theHuntForRedOctober, alecBaldwin).
by(jackRyan, patriotGames, harrisonFord).
by(cathyMuller, patriotGames, anneArcher).
by(jackRyan, clearAndPresentDanger, harrisonFord).
by(cathyMuller, clearAndPresentDanger, anneArcher).
by(president, airForceOne, harrisonFord).
by(frasierCrane, cheers, kelseyGrammer).
by(frasierCrane, frasier, kelseyGrammer).
by(rachelGreen, friends, jenniferAniston).
by(monicaGeller, friends, courteneyCox).
by(phoebeBuffay, friends, lisaKudrow).
by(ursulaBuffay, friends, lisaKudrow).
by(joeyTribbiani, friends, mattLeBlanc).
by(joeyTribbiani, joey, mattLeBlanc).
by(alexGarrett, joey, andreaAnders).
by(stephenColbert, dailyShow, stephenColbert).
by(stephenColbert, theColbertReport, stephenColbert).
by(addisonMontgomery, privatePractice, kateWalsh).
by(addisonMontgomery, greysAnatomy, kateWalsh).
by(mattMurdock, daredevil, benAffleck).
by(elektraNatchios, daredevil, jenniferGarner).
by(elektraNatchios, elektra, jenniferGarner).
by(elektraNatchios, elektra, lauraWard).
by(sydneyBristow, alias, jenniferGarner).
```

Responda às perguntas 11 a 13 **SEM** utilizar predicados de obtenção de soluções múltiplas (findall, setof e bagof), e **SEM** usar qualquer biblioteca do SICStus

Pergunta 11

Pontuou 0,200 de 1,000

Implemente o predicado **plays_twins(?Actor, ?Movie)**, que sucede se o ator **Actor** tiver interpretado pelo menos dois personagens diferentes no filme/série **Movie**, tipicamente indicativo de ter representado personagens gémeas.

```
by(Character, Movie, Actor),  
member(Character, Lista),  
length(Lista, L),  
L >= 2.
```

```
plays_twins(Actor, Movie):-  
    by(Char1, Movie, Actor),  
    by(Char2, Movie, Actor),  
    Char1 \= Char2.
```

% Nota: a adição de um cut nesta questão impediria a obtenção de resultados adicionais via backtracking

Pergunta 12

Pontuou 0,200 de 1,500

Implemente o predicado **actor_movies(+Actor, -Movies)** que devolve em *Movies* a lista de filmes/séries distintos protagonizados por *Actor* (a ordem dos elementos na lista não é relevante mas a lista não deve conter valores repetidos).

```
actor_movies(Actor, Movies) :-  
    by(_C, Movie, Actor),  
    append([Movie], [], Movies).
```

```
actor_movies(Actor, L):-  
    actor_movies(Actor, [], L).  
actor_movies(Actor, Acc, L):-  
    by(_Char, Movie, Actor),  
    \+ member(Movie, Acc), !,  
    actor_movies(Actor, [Movie|Acc], L).  
actor_movies(_Actor, Acc, Acc).
```

% Nota 1: Esta solução está no slide 3 do conjunto de slides PL5 (Collecting Solutions)

% Nota 2: Abordagem alternativa com assert/retract no slide 17 do conjunto de slides PL6 (Database Modification)

Pergunta 13

Pontuou 0,400 de 1,000

Implemente o predicado ***changeSelf/0*** que atualiza a base de conhecimento, substituindo factos em que a personagem e o ator são o mesmo para que a personagem seja indicada como *self*. Note que o predicado deve sempre suceder.

Exemplo:

```
| ?- by(X, dailyShow, Y).
X = stephenColbert, Y = stephenColbert
| ?- changeSelf.
yes
| ?- by(X, dailyShow, Y).
X = self, Y = stephenColbert
```

```
change_self :-
  clause(by(X, Movie, X), _),
  retractall(by(X, Movie, X)),
  asserta(by(self, Movie, X)).
```

```
changeSelf:-
  retract( by(Actor, Movie, Actor) ),
  assert( by(self, Movie, Actor) ),
  fail.
changeSelf.

% Nota 1: necessário repetir assert/retract para cada facto a remover (aqui implementado com um ciclo por falha)
% Nota 2: última cláusula garante que o predicado sucede sempre
```

Pergunta 14

Pontuação 1,500

Implemente o predicado ***playedBy(+Character, -List)***, que devolve em ***List*** uma lista com todos os atores que representaram a personagem ***Character***, contendo uma lista com os respectivos filmes/séries, conforme o exemplo abaixo (a ordenação da lista não é importante).

Exemplo:

```
| ?- playedBy(jackRyan, List).
List = [ benAffleck-[theSumOfAllFears], alecBaldwin-[theHuntForRedOctober], harrisonFord-[patriotGames, clearAndPresentDanger]]
```

```
playedBy(Character, List):-
  setof(Actor, Movie^by(Character, Movie, Actor), Actors),
  findall(A-Ms, ( member(A, Actors), findall(M, by(Character, M, A), Ms) ), List).
```

```
% Nota 1: o uso de setof garante que cada ator aparece apenas uma vez
% Nota 2: o uso do findall permite construir a lista de filmes para cada ator
```

Pergunta 15

Pontuação 2,000

Implemente o predicado **most_popular(+Exclude, -List, -NMovies)**, que devolve em **List** os nomes dos atores que participaram em mais filmes/séries diferentes (esta lista terá mais do que um valor em caso de empate; neste caso a ordem dos elementos não é importante), e em **NMovies** o número de filmes em que entraram. Não devem ser consideradas nos resultados os atores indicados na lista **Exclude**.

Exemplo:

```
| ?- most_popular([], List, NMovies).  
List = [harrisonFord,jenniferGarner], NMovies = 3  
  
| ?- most_popular([harrisonFord], List, NMovies).  
List = [jenniferGarner], NMovies = 3  
  
| ?- most_popular([jenniferGarner, harrisonFord], List, NMovies).  
List = [anneArcher,benAffleck,kateWalsh,kelseyGrammer,mattLeBlanc,stephenColbert], NMovies = 2
```

```
most_popular(Exclude, List, NMovies):-  
    setof(N-Actor, (_Char,_Movie,_Movies)^(  
        by(_Char, _Movie, Actor), \+ member(Actor, Exclude),  
        actor_movies(Actor, _Movies), length(_Movies, N) ), Templ),  
    last(Templ, NMovies-_),  
    findall(Actor, member(NMovies-Actor, Templ), List).
```

% Nota: o setof necessita de invocar o predicado by/3 para instanciar atores

Pergunta 16

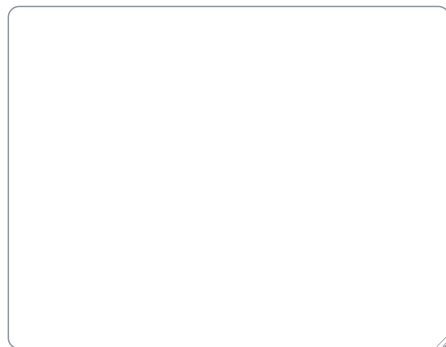
Pontuação 2,000

Implemente o predicado **connection_link(+Actor1, +Actor2, -ConnectionList)**, que determina uma ligação possível entre dois atores (se essa ligação existir). Caso não exista qualquer ligação entre os dois atores, o predicado deve falhar. Caso exista uma ligação, a lista **ConnectionList** deve ser preenchida com os nomes dos atores e filmes/séries que permitem ligar os dois atores **Actor1** e **Actor2**, conforme os exemplos abaixo.

Exemplos:

```
| ?- connection_link(jenniferAniston, courteneyCox, L).
L = [jenniferAniston, friends, courteneyCox] ? ;           % trabalharam juntas em friends
no

| ?- connection_link(jenniferGarner, bridgetMoynahan, L).
L = [jenniferGarner, daredevil, benAffleck, theSumOfAllFears, bridgetMoynahan] ? ;   % jenniferGarner trabalhou em daredevil com
benAffleck,
no                                           % que trabalhou em theSumOfAllFears com
bridgetMoynahan
```



```
connection_link(Actor1, Actor2, List):-
    connection_link(Actor1, Actor2, [Actor1], RList),
    reverse(RList, List).

connection_link(Actor1, Actor2, Tmp, [Actor2, Movie | Tmp]):- % ligação direta entre Actor1 e Actor2 via Movie
    by(_C1, Movie, Actor1),
    \+ member(Movie, Tmp),
    by(_C2, Movie, Actor2),
    Actor1 \= Actor2.

connection_link(Actor1, Actor2, Tmp, List):- % se não há ligação direta, seguir via outro ator
    by(_C1, Movie, Actor1),
    \+ member(Movie, Tmp),
    by(_C2, Movie, Actor3),
    Actor1 \= Actor3,
    \+ member(Actor3, Tmp),
    connection_link(Actor3, Actor2, [Actor3, Movie | Tmp], List).
```

% Nota: implementação usando pesquisa em profundidade

Pergunta 17

Pontuação 1,000

Implemente o predicado ***pretty_print(+ConnectionList)***, que recebe uma lista de atores e filmes/séries no formato gerado pela questão anterior, e imprime a informação para a consola no formato que se pode ver no exemplo abaixo (note as pontuações e espaçamentos).

Exemplo:

```
| ?- connection_link(lauraWard, bridgetMoynahan, L), pretty_print(L).
lauraWard worked in elektra with jenniferGarner,
  who worked in daredevil with benAffleck,
  who worked in theSumOfAllFears with bridgetMoynahan.
L = [lauraWard, elektra, jenniferGarner, daredevil, benAffleck, theSumOfAllFears, bridgetMoynahan] ?
no
```

```
pretty_print([A, B, C|T]):-
    write(A), write(' worked in '), write(B), write(' with '), write(C),
    pretty_print2(T).

pretty_print2([]):-
    write('.'), nl.

pretty_print2([B, C|T]):-
    write(','), nl, write(' who worked in '), write(B), write(' with '), write(C),
    pretty_print2(T).
```

◀ PL9 - Applications and Libraries

SICStus Prolog ▶